

Mikiri: Knowing When a Frozen Go Engine Has Searched Enough

Anonymous submission

Abstract

A strong Go player glances at a forced recapture and plays it. The same player may sit for ten minutes over a fight that decides the game. Engines in the AlphaZero family do neither: they give every move the same search budget, and that is how they are compared. We ask how much that habit costs, and how much of it a frozen engine can recover with its network and its search left exactly as they are. Mikiri wraps KataGo and climbs a short ladder of search budgets, stopping at the first rung where the winrate still at stake no longer pays for the next rung. To build and test it we searched 6,000 positions at seven budgets and scored every candidate move with an independent search, which gives a yardstick that the engine cannot grade itself on. Offline, Mikiri matches uniform search at 1.8 to 2.1 times the visits, ahead of ten published stopping rules re-implemented on the same engine. In play against KataGo at the same visits it gains +228 Elo over 1,000 games (95% interval +203 to +254), and it draws level with KataGo given twice its visits. Doubling KataGo’s own budget is worth +237, and the strongest prior rule, played the same way, gains +135. One small idea does most of the work: divide the predicted stake by the price of the next rung. It turns stopping signals that are worse than no gate into gains, and it improves other people’s rules too. We also report what did not work: similar positions do not share answers, and an exact memory of past searches saves visits but almost no network evaluations.

1 Introduction

Watch a strong player and you will see two kinds of move. Some are played almost before the opponent’s stone has settled: a recapture, a connection, the only move. Others get long, visible thought. The player is not being inconsistent. They are spending attention where the game can still change.

Engines in the AlphaGo line [Silver *et al.*, 2016; Silver *et al.*, 2017] do not make that distinction. They choose a move with a tree search guided by a policy network, which proposes moves, and a value network, which judges positions.

The search is given a fixed number of *visits*, which count how many times it descends the tree, and strength is measured by how well it plays at that number. KataGo [Wu, 2019], the strongest engine anyone can download, follows the convention. At 200 visits per move, a forced recapture and a whole-board fight get 200 visits each.

How much does that cost? The question is older than neural engines. Time management for Monte-Carlo tree search was studied when the engines were weaker and the gains from thinking longer were larger [Huang *et al.*, 2010; Baier and Winands, 2016]. For AlphaZero-style agents, the evidence that a per-move budget helps comes from 9×9 boards and small networks [Lan *et al.*, 2021; Ye *et al.*, 2022; Muppidi *et al.*, 2026], and KataGo already adapts its search inside the tree [KataGo project, 2026]. It is reasonable to suspect that a modern engine on the full board has little left to gain. We did not know, and we found no measurement.

So we built one. Mikiri is a wrapper that changes nothing about KataGo except how many visits a move receives. It searches a position at 50 visits and asks a small model whether the decision is settled. If not, it searches at the next rung of a ladder, 200 visits, and asks again. The model predicts how much winrate the current choice might still be giving up, and the search stops when that stake, divided by the visits the next rung would cost, falls below a threshold. The name is a Japanese word for the judgment that one has seen enough.

Measuring this properly turned out to be most of the work, because a search cannot grade its own choices. Our testbed scores every candidate move with a separate search that is forced to play it, counts compute three ways, and plays both sides with the same engine and network. Along the way we tested the idea this project started from, retrieving analyses of similar positions, and found that it does not help a frozen engine. We report that too.

In summary, this paper contributes:

1. **A testbed with exact accounting.** Both players use the same engine, network and move-selection rule. Compute is reported as visits, as visits with every restarted search charged, and as network evaluations counted per player (Section 3).
2. **A ladder dataset with independent labels.** 6,000 positions searched at seven budgets, every chosen move scored by a forced search. It shows where search error

lives, and it predicts match results before the games are played (Sections 4 and 7).

3. **A stopping rule worth nearly a doubling of search.** +228 Elo over KataGo at equal visits, a draw against KataGo at twice the visits, and +93 Elo with 20% fewer (Section 7).
4. **A comparison with ten published rules** on one engine, offline for all of them and in play for the two strongest (Sections 6.1 and 7).
5. **The rate rule.** Pricing the next rung matters more than the model that predicts the stake, and it improves prior rules as well (Section 6).
6. **Two negative results** on what a frozen engine can retrieve (Section 8).

2 Related Work

Deciding how long to search. Huang *et al.* [2010] extend a search when the most-visited move is not the one with the highest value, a sign that the search has not made up its mind. Baier and Winands [2016] compare several such strategies across five games and find that stopping early, once the leader can no longer be overtaken, is the most reliable; Leela Chess Zero ships a version of that rule together with a stop on how much the visit distribution is still changing [Leela Chess Zero contributors, 2026]. Hay *et al.* [2012] and Kaufmann and Koolen [2017] derive stopping rules from the value of information and from best-arm identification. For AlphaZero-style agents, DS-MCTS [Lan *et al.*, 2021] trains networks to predict whether the current choice will survive more search, V-MCTS [Ye *et al.*, 2022] stops when virtually expanded visit distributions have converged, and Muppidi *et al.* [2026] choose a budget before the search starts. These neural results come from 9×9 boards or small planners. We re-implement all of them on 19×19 with a strong network and test them side by side (Section 6.1).

Reusing search. Transposition tables and graph search share statistics between identical positions inside one search [Childs *et al.*, 2008; Czech *et al.*, 2021], and opening books do so across games [Wu, 2026]. Retrieving similar positions helps agents whose networks are trained to read their neighbours [Humphreys *et al.*, 2022; Goyal *et al.*, 2022], and memory can help inside a single search [Xiao *et al.*, 2018]. Our question is narrower: what can an engine reuse when its network is fixed?

3 Testbed

Every number in this paper comes from one harness, built so that the two players differ only in how they spend visits.

Engine and rules. Both players query KataGo v1.18.2 through its analysis engine. Each query is a fresh search with an exact visit limit and one search thread, so no player inherits a tree from an earlier move and a visit means the same thing at every budget. The network is the 18-block b18c384nbt unless stated. Games use area scoring, positional superko and komi 7, the value at which the engine rates the empty board as even. Both players pick moves the way KataGo does by default, sampling in proportion to visits^{1/T} with a temperature

T that decays from 0.5 to 0.1 over the game. A player resigns after three consecutive turns below 3% winrate.

Three ways to count compute. This needs care, because Mikiri restarts its search. A position searched at 50 visits and then at 200 has been asked for 250 visits of work; whether it has done 250 visits of work depends on the engine. With one thread and cached network evaluations the 50-visit tree is a prefix of the 200-visit tree, and we measured this: on fresh engines, a 200-visit search followed by a 400-visit search evaluated 366 to 396 network rows, against 353 to 395 for the 400-visit search alone. We nonetheless report three numbers for every match and let the reader choose: *visits*, the size of the deepest search per move, which is what an in-engine implementation would pay; *restart visits*, the sum of every search requested, an upper bound; and *evaluations*, the network rows each player’s own engine process actually executed, read from its log. Budgets are matched on visits.

Matches. A match is a series of games from the empty board with alternating colours. Confidence intervals are bootstrap percentiles over colour-balanced pairs of games, and Elo is $-400 \log_{10}(1/s - 1)$ for a score s . As a check on the harness, KataGo against itself at 200 visits scores 50.2% over 400 games (+2 Elo).

4 The Ladder Dataset

Before building a stopping rule we wanted to know what one could gain, position by position. We played 300 self-play games at budgets drawn log-uniformly from 30 to 200 visits and sampled 20 positions per game, which gives 6,000 positions from the distribution a low-budget player actually meets. Each position is searched at 50, 100, 200, 400, 800, 1,600 and 3,200 visits.

Here it gets a little tricky. The value a search reports for its own choice is exactly the estimate that might be wrong, so it cannot be the yardstick. Instead, for every distinct move that any rung chose, we run a separate 1,600-visit search restricted to that move at the root. Its winrate $q(s, a)$ is our value for playing a in position s . Every move is scored the same way, by a search that did not choose it. The *regret* of stopping at rung k is $r_k(s) = \max_a q(s, a) - q(s, a_k(s))$, where a_k is the move chosen at rung k and the maximum runs over scored moves. Regret is in winrate, so it is small in positions that are already decided and large where the game can still turn, which is the behaviour we want from a stake.

To compare policies we convert regret back into visits. For a policy with mean regret $\bar{r}$ we find the uniform budget with the same mean regret, interpolating the uniform curve in log visits, and divide it by the policy’s own mean cost. We call this ratio the *compute multiplier*: how many times more visits uniform search needs to do as well.

Regret is rare and concentrated. At 200 visits the engine gives up 0.99% of winrate per move on average. That average hides the shape. 71% of positions carry no regret at all, and the worst tenth hold 84% of the total (Figure 1). Regret also depends on where in the game you are: 0.19% per move over the first 40 moves and 1.4 to 1.6% after move 100. A stopping rule that knew the true regret, and stopped at the first rung with none, would match uniform search at twelve times the visits



Figure 1: Left: share of all regret held by the worst positions, at three budgets. Right: mean regret by move number.

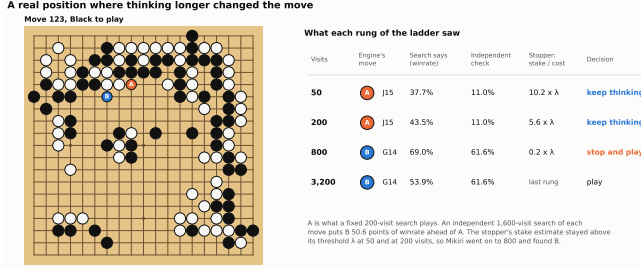


Figure 2: A position where the stopper keeps searching and the move changes. The threshold λ shown gives a mean cost of 200 visits.

for a 200-visit budget. That is the room available.

Figure 2 shows what one of the costly positions looks like. At 50 and at 200 visits the engine chooses a move that an independent search rates at 11%. At 800 visits it finds one rated at 62%. The game turns on whether the search is allowed to continue.

No single signal finds it. The natural first attempt is to read one statistic off the search. Table 1 asks how well each one ranks positions by whether more than 0.5% of winrate is still on the table. Measures of how contested the root is, such as the top move’s share of visits, reach an AUC of about 0.74. KataGo’s own prediction of its short-term winrate error, which it uses inside the tree, reaches only 0.58 to 0.60 as a predictor of whether the root decision will improve. The classical cue, that the most-visited move is not the highest-valued one [Huang *et al.*, 2010], reaches 0.59 to 0.61. The reason is plain once you look at the positions: contested roots are common, and most are contested between moves of nearly equal value, where being wrong costs nothing.

5 Mikiri

Figure 3 shows how Mikiri decides one move. The parts are a ladder of budgets, a model that predicts the stake, a rule that prices it, a ledger that keeps the total honest, and a memory.

The rate rule. Consider what the stopping decision is actually trading. Let the ladder have rungs $v_1 < \dots < v_m$, and let a stopping policy π stop position s at rung $\pi(s)$. We want the policy that gives up the least winrate for a fixed average cost,

$$\min_{\pi} \mathbb{E}[r_{\pi(s)}(s)] \quad \text{s.t.} \quad \mathbb{E}[v_{\pi(s)}] \leq B. \quad (1)$$

Put a multiplier λ on the constraint and the problem falls apart

Signal	50	200	800
Top move’s visit share	0.75	0.74	0.74
LCB margin	0.74	0.73	0.73
v1 entropy gate	0.73	0.73	0.74
KL(visits, prior)	0.65	0.63	0.61
Most-visited is not best-valued	0.60	0.59	0.61
KataGo’s predicted winrate error	0.58	0.58	0.60
Searched minus raw winrate	0.57	0.56	0.56

Table 1: AUC for ranking positions by remaining regret above 0.5%, from the search at 50, 200 and 800 visits.

into one decision per position: each minimises $r_k(s) + \lambda v_k$. At rung j the visits already spent are gone, so continuing one more rung is worth it exactly when $\mathbb{E}[r_j - r_{j+1} \mid x] > \lambda(v_{j+1} - v_j)$, where x is whatever the finished search shows. The reduction $r_j - r_{j+1}$ can be at most r_j . Replace it by a prediction $\hat{r}_j(x)$ of the regret remaining, and the rule becomes: stop when

$$\frac{\hat{r}_j(x)}{v_{j+1} - v_j} < \lambda. \quad (2)$$

One λ prices every decision on the ladder alike. On a geometric ladder the next rung costs more each time, so going from 800 to 3,200 visits needs four times the stake that going from 200 to 800 does. A flat threshold on $\hat{r}_j$ ignores that price and, as we will see, spends most of a long ladder’s budget on a handful of positions that no amount of search resolves.

The model. $\hat{r}$ is a gradient-boosted ensemble of 150 trees of depth 3, fitted to the square root of regret with all non-final rungs pooled. Its 30 inputs come from the search that just finished, at no extra engine cost: how contested the root is (visit shares, entropy, the margin between the best move’s lower confidence bound and its nearest rival), how the search disagrees with the network (priors, the KL divergence from prior to visits, raw against searched winrate), what is at stake (winrate, its distance from 50%, score), KataGo’s own raw predictions of its error, the game phase, and what changed since the previous rung. Every offline number we report is cross-fitted by game, so no position is scored by a model that saw its game.

The ledger. Mikiri is granted the baseline’s budget B for every move, and a ledger records what it spends. The offline frontier gives a map $\lambda(c)$ from a target mean cost c to a threshold. After each move the controller sets the next target to B plus the ledger’s balance spread over a horizon of 3,000 moves, and divides by a running ratio of realised to commanded spending, which corrects for the difference between the dataset and real games. Spending tracks the grant to within 1% in every match we report.

Memory. Finished searches from the first 80 moves are stored under a key that is invariant to the eight board symmetries and to move order. When a later game reaches the same position, Mikiri plays from the stored search and spends nothing.

6 Offline Results

Multiplier. On the four-rung ladder (50, 200, 800, 3,200) the stopper at a mean cost of 200 visits gives up as little winrate

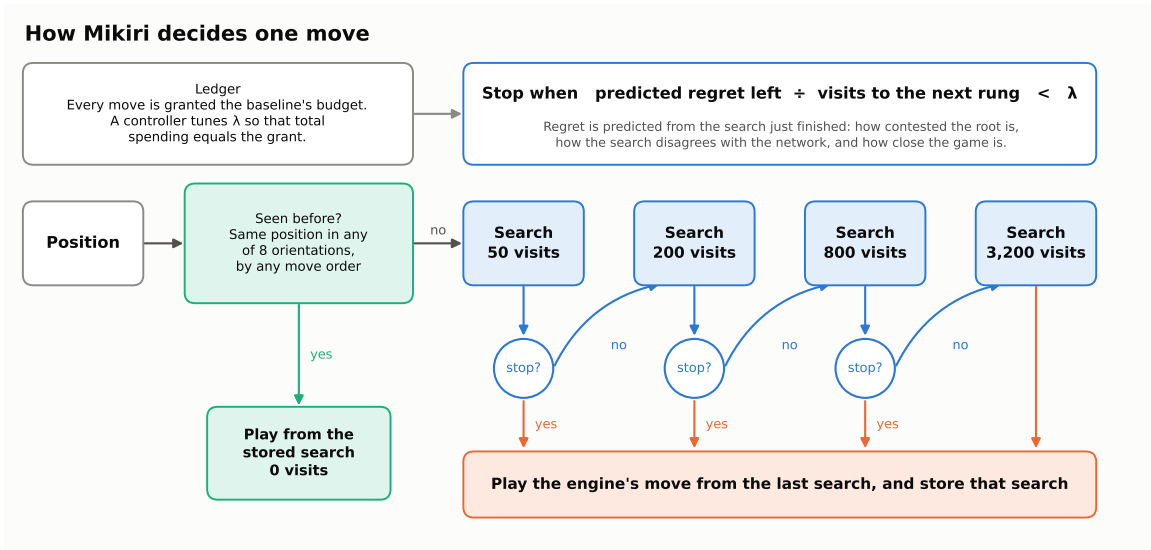


Figure 3: How Mikiri decides one move.

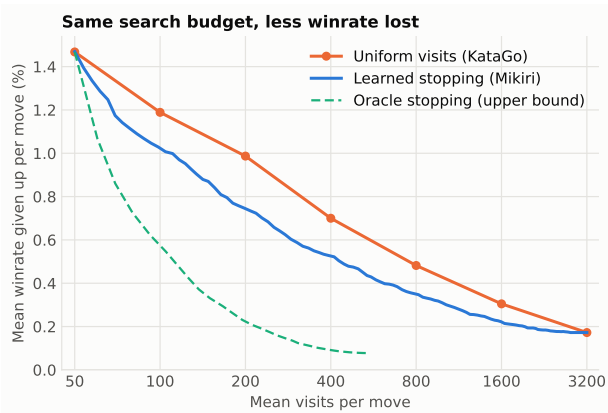


Figure 4: Mean regret against mean visits on the four-rung ladder, for uniform search, the learned stopper, and an oracle that knows the true regret.

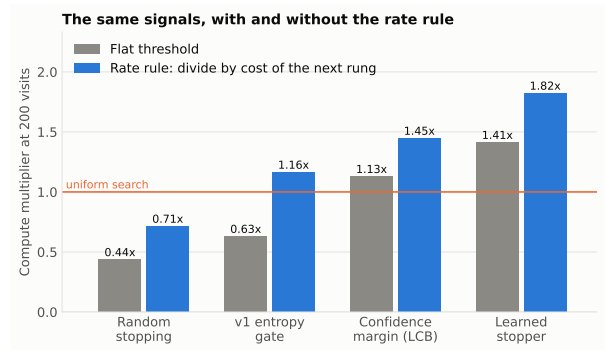


Figure 5: Four stopping signals on the four-rung ladder at 200 visits, with a flat threshold and with the rate rule.

as uniform search at 358, a multiplier of 1.80, and it stays between 1.69 and 1.88 for budgets from 100 to 800 visits (Figure 4). If every restarted search is charged in full, so that reaching the 800 rung costs $50 + 200 + 800$, the multiplier is 1.29 to 1.51. On the full doubling ladder it reaches 2.13.

What matters, and what does not. Removing the features that track change between rungs, removing KataGo’s raw error predictions, or keeping only seven features changes the multiplier at 200 visits by at most 0.09. Three hundred labeled positions already give 1.50. Two choices do matter. Fitting the square root of regret rather than regret raises the multiplier from 1.74 to 1.82, because a few very large blunders otherwise dominate the fit. Replacing the rate rule with a flat threshold lowers it to 1.41. Figure 5 shows the same contrast for handwritten signals: with a flat threshold, the entropy gate of an earlier prototype is worse than no gate at all (0.63), and with the rate rule the same signal is a gain (1.16).

6.1 Comparison with Prior Rules

How does this compare with what has been published? We re-implemented ten stopping rules from their papers and source code and ran them on the same data, where every rule sees the same root statistics and is scored by the same forced values (Table 2). *BEHIND* and *UNST* search longer when the player is losing or when the most-visited move is not the highest-valued one [Huang *et al.*, 2010]. *STOP* ends a search once the runner-up cannot overtake the leader in the visits that remain, or in a fraction p of them, and *CLOSE* extends it when the top two moves are close [Baier and Winands, 2016]. *Smart pruning* is Leela Chess Zero’s form of *STOP*, shipped with a stop on the per-visit KL gain of the visit distribution [Leela Chess Zero contributors, 2026]. The *VOI* rule stops when a bound on the value of further samples falls below their cost [Hay *et al.*, 2012], and the *BAI* rule when the leader’s confidence interval separates from every rival’s [Kaufmann and Koolen, 2017]. The *state-only* rule predicts regret from what is known before any search, in the spirit of Muppidi *et al.* [2026]. Two learned rules are adaptations, and we do not claim they reproduce the

Rule	As published, 200	As published, 400	With rate rule, 200
VOI stop [Hay <i>et al.</i> , 2012]	0.34 [0.30, 0.40]	0.24 [0.20, 0.31]	1.05 [0.86, 1.21]
BAI stop [Kaufmann and Koolen, 2017]	0.77 [0.63, 1.01]	0.59 [0.50, 0.71]	1.18 [1.09, 1.29]
BEHIND [Huang <i>et al.</i> , 2010]	0.89 at 174	0.89 at 348	–
STOP, $p = 0.45$ [Baier and Winands, 2016]	1.25 at 134	1.19 at 512	–
Smart pruning [Leela Chess Zero contributors, 2026]	1.17 at 159	1.21 at 308	–
CLOSE [Baier and Winands, 2016]	1.15 at 229	1.11 at 453	–
UNST [Huang <i>et al.</i> , 2010]	1.21 at 239	1.17 at 475	–
KLD gain [Leela Chess Zero contributors, 2026]	1.32 [1.18, 1.49]	1.23 [1.02, 1.42]	–
State only [Muppidi <i>et al.</i> , 2026]	1.18 [0.97, 1.40]	1.02 [0.89, 1.23]	1.54 [1.35, 1.74]
DS-MCTS [Lan <i>et al.</i> , 2021]	1.48 [1.28, 1.73]	1.48 [1.29, 1.73]	1.75 [1.60, 1.92]
V-MCTS [Ye <i>et al.</i> , 2022]	1.60 [1.44, 1.76]	1.54 [1.36, 1.70]	–
LCB margin	1.14 [0.95, 1.36]	0.82 [0.70, 0.97]	1.45 [1.24, 1.70]
Mikiri	1.40 [1.21, 1.63]	1.35 [1.15, 1.61]	2.13 [1.88, 2.41]

Table 2: Compute multipliers on the doubling ladder (above 1 is better than uniform search), with 95% bootstrap intervals over games. Threshold rules are swept and shown at their best point for each budget; rules without a threshold are shown at the configuration nearest the budget, with its cost in visits.

authors’ systems: for DS-MCTS we train a classifier on root statistics to the authors’ label $U(s, n)$, in place of their board-input networks, and for V-MCTS we run the authors’ virtual expansion on KataGo’s root priors, visits and winrates with their published $r = 0.2$ and $\epsilon = 0.1$. Every rule is charged continuation cost, which is what a rule inside the engine pays. A reader who prefers to charge Mikiri for every restart should compare its four-rung figure of 1.44 at 200 visits with the 1.60 of V-MCTS, which stops inside one search and has nothing to restart. The matches in Section 7 settle that question in network evaluations.

Three things stand out. First, the two rules with a statistical stopping guarantee are far worse than uniform search here (0.34 and 0.77 at 200 visits). They stop on confidence in the leading move, and confidence in the leader says little about whether the game is still in doubt. Second, V-MCTS is the strongest prior rule, at 1.60, and Mikiri exceeds it by 0.54 [0.28, 0.84] at 200 visits and 0.35 [0.09, 0.65] at 400, as paired differences over the same resampled games. Third, the rate rule improves every signal it can be applied to. It lifts the DS-MCTS classifier from 1.48 to 1.75, and at 400 visits that combination ties Mikiri (0.03 [-0.16, 0.27]). The price on the next rung matters more than which learned signal it is attached to.

What to predict, and how to price it. Remaining regret is an upper bound on what continuing can recover, so one might ask whether the model should predict something closer to the derivation. Table 3 tries the alternatives it names: the gain from the next rung, the gain from the whole ladder, and the best recovery per visit over any continuation, $\max_{k>j}(r_j - r_k)/(v_k - v_j)$. With perfect information that last index reproduces the hindsight optimum (14.11 against 14.16), so it is the right quantity in principle. Learned from search statistics it trails remaining regret under the rate rule (1.63 to 1.87 against 1.83 to 2.14). The next-rung gain is weakest on both counts: myopic as an oracle, because it stops whenever one more rung does not help even if two more would, and noisy as a target. Pricing remaining regret by the visits to the top of the ladder is worse than no price at all. The rate rule works because it pairs a target that search statistics

Scored	Divided by	Oracle,	Learned, 200 visits		
		100 visits	4	6	7 rungs
r_j	1 (flat)	5.99	1.41	1.56	1.51
r_j (rate rule)	$v_{j+1} - v_j$	8.69	1.83	1.98	2.14
r_j	$v_m - v_j$	5.86	1.35	1.50	1.39
$r_j - r_{j+1}$	$v_{j+1} - v_j$	4.96	1.50	1.54	1.44
$r_j - r_m$	$v_{j+1} - v_j$	9.07	1.79	1.98	2.04
$\max_{k>j} \frac{r_j - r_k}{v_k - v_j}$	built in	14.11	1.63	1.87	1.77
Hindsight optimum		14.16	–	–	–

Table 3: Multipliers for different quantities to score at rung j and prices to divide them by (m is the top rung). Oracle: true quantity, four-rung ladder, 100 visits. Learned: the same model fitted to each quantity, 200 visits, three ladders.

can predict with the one cost that is known exactly.

Do the prior rules see something we miss? If the hand-derived statistics behind those rules carried information the 30 features lack, feeding them to the regressor should help. We added the KLD gain, the VOI bound, the BAI gap and the V-MCTS distance as extra inputs, one at a time and all together. The multiplier at 200 visits moved from 2.14 to between 2.07 and 2.17, and no paired difference was distinguishable from zero in the direction of a gain. The rules differ in what they do with the root statistics; the statistics themselves are used up.

7 Matches

Offline numbers are only a forecast. Here is what happened when the games were played.

Equal visits. With the six-rung ladder (50, 200, 400, 800, 1,600, 3,200) Mikiri scores 78.8% (750–175–75), a gain of +228 Elo (+203 to +254). With the four-rung ladder it gains +206 (+181 to +232) over 1,000 games. For scale, KataGo given twice the visits gains +237 (Table 4). We then played that comparison directly: Mikiri at 200 visits against KataGo at 400. The result is level, 268–271–61 over 600 games (–2 Elo, –27 to +24), with Mikiri running 102 network evaluations per move to KataGo’s 143.

Player (vs. KataGo at 200 visits)	Games	W–L–D	Elo (95% interval)	Visits	Restart	Evals
KataGo, 100 visits	400	77–301–22	−220 (−262, −181)	100	100	40 : 71
KataGo, 283 visits	400	247–126–27	+108 (+76, +143)	283	283	98 : 73
KataGo, 400 visits	400	310–73–17	+237 (+198, +280)	400	400	132 : 73
Mikiri, seven-rung ladder			(running)			
Mikiri, six-rung ladder	1,000	750–175–75	+228 (+203, +254)	201	304	102 : 78
Mikiri, four-rung ladder	1,000	728–196–76	+206 (+181, +232)	200	251	99 : 74
Mikiri, four-rung, 175-visit grant	600	432–129–39	+193 (+163, +225)	175	217	89 : 76
Mikiri, four-rung, 160-visit grant	1,000	586–325–89	+93 (+72, +114)	160	197	80 : 74
Mikiri, four-rung, 140-visit grant			(running)			
Mikiri, four-rung, no move sampling	800	561–183–56	+178 (+153, +205)	200	250	115 : 96
V-MCTS rule (Ye et al.)	600	395–173–32	+135 (+108, +164)	200	350	87 : 72
Mikiri’s rule on the V-MCTS ladder	600	404–160–36	+150 (+121, +180)	200	350	92 : 74
DS-MCTS-style rule (Lan et al.)	600	395–157–48	+146 (+117, +176)	200	351	105 : 77
LCB margin with rate rule, no learning	600	349–201–50	+88 (+61, +115)	200	250	100 : 75
Entropy gate, flat threshold	400	82–291–27	−201 (−244, −161)	205	257	113 : 75
Memory only	600	276–292–32	−9 (−37, +19)	158	158	69 : 69
Mikiri four-rung + memory	1,200	908–219–73	+227 (+204, +250)	201	255	109 : 76

Table 4: Matches against KataGo at 200 visits per move, 19×19, same network. *Visits* is the mean size of the deepest search per move, *Restart* charges every search requested, and *Evals* is network evaluations per move for the player and for its opponent.

Fewer visits, and the strictest count. A fixed-budget engine finds much of its next tree already cached, and Mikiri’s searches vary in size, so less carries over and it runs more evaluations per visit. We therefore also held it below the opponent. Granted 160 visits, Mikiri requests fewer visits than KataGo even with every restart charged (197 against 200) and still gains +93 Elo (+72 to +114). Counting evaluations, the cleanest comparison is with KataGo at 283 visits, which runs 98 per move and gains +108; Mikiri’s six-rung ladder runs 102 and gains +228. Figure 6 places every match against the uniform curve under all three counts.

Is it the sampling? A search that stops when one move dominates has sharper visit counts than a fixed-budget search, and KataGo samples among well-visited moves early in the game, so one could worry that Mikiri benefits from sampling rather than from better search. To rule that out we drew 400 balanced openings, played each twice with colours swapped, and had both players always play their top move. Mikiri gains +178 Elo (+153 to +205).

Did the dataset predict this? Take the cross-fitted regret at the cost Mikiri actually used, convert it to equivalent uniform visits, and read that budget’s Elo off the measured uniform curve. This forecasts +197 for the four-rung match (measured +206), +126 at 160 visits (+93), +225 for the six-rung ladder (+228), +145 for the V-MCTS rule (+135) and +128 for the DS-MCTS-style rule (+146). Each is within 35 Elo, and both prior rules are placed below Mikiri’s equal-visit matches, as measured. The forecast has one clear miss: Mikiri’s rule on the short V-MCTS ladder, forecast at +196 and measured at +150 (Table 5). The ladder dataset is a useful guide for choosing among rules and no substitute for playing the games.

Prior rules in play. We packaged the two strongest prior rules into the same player, under the same controller, at the same 200 visits. The V-MCTS rule gains +135 Elo (+108 to +164) and the DS-MCTS-style classifier +146 (+117 to +176). The DS-MCTS-style rule runs 105 evaluations per

Rule	Visits	Equiv.	Forecast	Measured (95%)
Mikiri, 4 rungs	160	297	+126	+93 (+72, +114)
Mikiri, 4 rungs	175	325	+160	+193 (+163, +225)
Mikiri, 4 rungs	200	360	+197	+206 (+181, +232)
Mikiri, 6 rungs	201	387	+225	+228 (+203, +254)
Mikiri, short ladder	200	358	+196	+150 (+121, +180)
V-MCTS	200	312	+145	+135 (+108, +164)
DS-MCTS-style	200	299	+128	+146 (+117, +176)

Table 5: Elo forecast from the ladder dataset against Elo measured in play, versus KataGo at 200 visits.

move, more than Mikiri’s six-rung ladder, so Mikiri is ahead of it however compute is counted. V-MCTS is the more interesting case. At equal visits Mikiri is ahead with intervals that do not overlap, but V-MCTS never searches past 400 visits, so more of its work is already in the cache: it runs 87 evaluations per move to Mikiri’s 99. Two more matches say what that means. Mikiri’s stopper on V-MCTS’s own ladder (50, 100, 200, 400), so that the rules differ in nothing else, gains +150 (+121 to +180) at 92 evaluations, which the match cannot separate from V-MCTS. Mikiri’s four-rung ladder granted 175 visits, so that its evaluations match, gains +193 (+163 to +225) at 89 evaluations to V-MCTS’s +135 at 87: a lead of about 60 Elo whose intervals touch. So the lead at equal visits comes from ladders that reach 3,200 visits, which the rate rule makes affordable and V-MCTS does not exploit; those deep searches cost more evaluations per visit, and per evaluation the lead is smaller. For completeness, the LCB margin with the rate rule and no learning gains +88 (+61 to +115), and the entropy gate with a flat threshold −201 (−244 to −161).

Where the visits go. Nobody told Mikiri when to think. It uses a third of the budget over the first 20 moves, where the dataset found almost no regret to recover, and about 270 visits per move around move 140. After move 60 it spends about

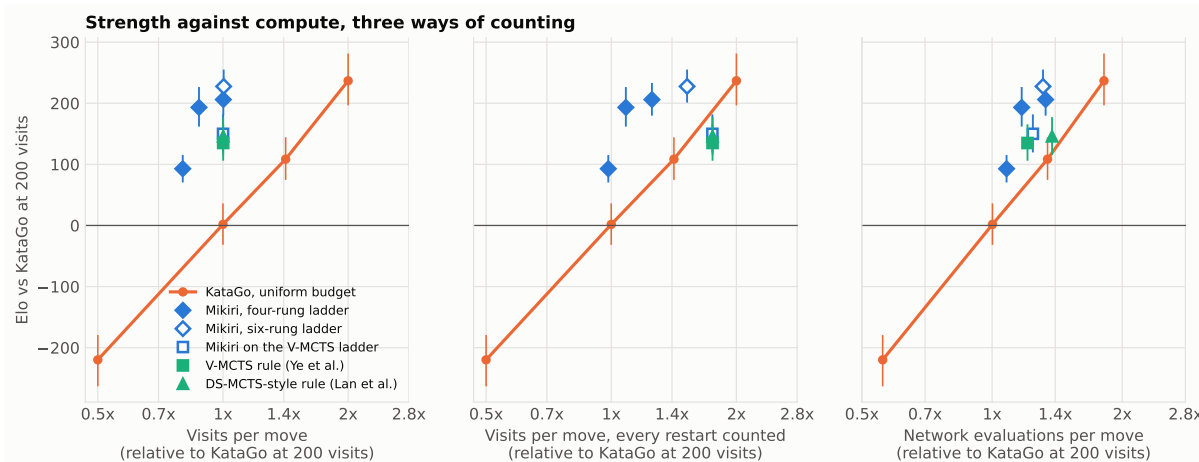


Figure 6: Elo against KataGo at 200 visits as a function of compute relative to that baseline, counted three ways. The line is KataGo at uniform budgets; bars are 95% intervals.

Opponent visits	Games	W-L-D	Elo (95% interval)
100	400	262-119-19	+130 (+95, +167)
400			(running)
800			(running)
400 (Mikiri at 200)	600	268-271-61	-2 (-27, +24)
200, b28 network			(running)
200, 13×13			(running)
200, 9×9			(running)

Table 6: The four-rung stopper, trained once on 19×19 b18 positions, against KataGo at matched visits in other settings.

130 visits when it rates its winrate above 90% and up to 330 when it is behind but not lost. It scores 76.2% as Black and 77.0% as White. All of this follows from predicting regret in winrate units, and none of it was programmed.

Other settings. The same model, trained once and never retrained, was played in further settings (Table 6).

8 What a Frozen Engine Can Retrieve

This project began with a different idea: store deep analyses, find similar positions with a nearest-neighbour index, and reuse what was found. We tested it, and we tested the honest version of it.

Exact memory. Positions recur more than one might guess. Across 300 self-play games, a position at move 20 had occurred in an earlier game, up to symmetry, half of the time. In play, memory alone served 21.2% of moves over 600 games at 158 visits per move, with strength unchanged (−9 Elo, −37 to +19), which is what should happen when a hit returns the search that would have been run. It did not reduce network evaluations (69 per move against 69). KataGo caches evaluations for the life of its process, so the positions that memory serves were already cheap.

Approximate retrieval. We embed positions with the raw policy and ownership maps, align over the eight symmetries, and offer the nearest stored position’s 3,200-visit move as a

proposal. Where the 200-visit and 3,200-visit moves differ (1,605 positions), the nearest neighbour’s move is right 5.5% of the time. The policy network’s own best alternative is right 43% of the time. A neighbour’s move matches in 96% of cases at similarity above 0.999, which in practice means the same position, and in 27% at similarity between 0.90 and 0.97. Neighbours’ regret predicts a position’s regret with an AUC of 0.55. Go is unforgiving here: a stone’s difference changes the answer, so proximity in a smooth embedding is weak evidence about the best move.

9 Limitations

Our results are for KataGo networks at 100 to 800 visits, against KataGo, a range where a doubling of visits is worth about 230 Elo. At tournament budgets of tens of thousands of visits the curve is flatter and we have not measured what remains. Regret is defined by a search and inherits its blind spots, though the matches do not depend on it. Of the regret that survives a 200-visit search, 57% sits on moves that the search gave under 10% of its visits, which bounds what any rule that reads root statistics can recover. Our versions of prior rules are re-implementations on a different engine than their authors used. And a stopping rule built on the engine’s own statistics offers no protection against adversarial policies [Wang *et al.*, 2023]: it would stop early, with full confidence, in exactly the positions those attacks construct.

10 Conclusion

A frozen Go engine leaves most of a doubling of search on the table by spending visits uniformly. A small learned rule recovers it, and the part of the rule that matters most is the price it puts on continuing. We think the more general lesson is about measurement: a search cannot grade itself, and once a yardstick independent of the search exists, questions that looked settled turn out to have numbers attached.

Ethical Statement

There are no ethical issues.

Use of AI Assistance

A large language model coding assistant was used extensively in this work: to implement the testbed and analysis code, to run and monitor the experiments, to produce the figures, and to draft and edit the text. The authors directed the work and are responsible for all of its content. Every reported number is produced by released scripts from released data and game records.

References

[Baier and Winands, 2016] Hendrik Baier and Mark H. M. Winands. Time management for Monte Carlo tree search. *IEEE Transactions on Computational Intelligence and AI in Games*, 8(3):301–314, 2016.

[Childs *et al.*, 2008] Benjamin E. Childs, James H. Brodeur, and Levente Kocsis. Transpositions and move groups in Monte Carlo tree search. In *IEEE Symposium on Computational Intelligence and Games*, pages 389–395, 2008.

[Czech *et al.*, 2021] Johannes Czech, Patrick Korus, and Kristian Kersting. Improving AlphaZero using Monte-Carlo graph search. In *International Conference on Automated Planning and Scheduling*, pages 103–111, 2021. arXiv:2012.11045.

[Goyal *et al.*, 2022] Anirudh Goyal, Abram L. Friesen, Andrea Banino, Theophane Weber, Nan Rosemary Ke, Adrià Puigdomènech Badia, Arthur Guez, Mehdi Mirza, Peter C. Humphreys, Ksenia Konyushkova, Laurent Sifre, Michal Valko, Simon Osindero, Timothy Lillicrap, Nicolas Heess, and Charles Blundell. Retrieval-augmented reinforcement learning. In *International Conference on Machine Learning*, pages 7740–7765, 2022. arXiv:2202.08417.

[Hay *et al.*, 2012] Nicholas Hay, Stuart Russell, David Tolpin, and Solomon Eyal Shimony. Selecting computations: Theory and applications. In *Uncertainty in Artificial Intelligence*, 2012. arXiv:1207.5879.

[Huang *et al.*, 2010] Shih-Chieh Huang, Rémi Coulom, and Shun-Shii Lin. Time management for Monte-Carlo tree search applied to the game of Go. In *International Conference on Technologies and Applications of Artificial Intelligence*, pages 462–466, 2010.

[Humphreys *et al.*, 2022] Peter C. Humphreys, Arthur Guez, Olivier Tieleman, Laurent Sifre, Théophane Weber, and Timothy Lillicrap. Large-scale retrieval for reinforcement learning. In *Advances in Neural Information Processing Systems*, 2022. arXiv:2206.05314.

[KataGo project, 2026] KataGo project. Other methods implemented in KataGo. <https://github.com/lightvector/KataGo/blob/master/docs/KataGoMethods.md>, 2026. Accessed 2026-10-01.

[Kaufmann and Koolen, 2017] Emilie Kaufmann and Wouter M. Koolen. Monte-Carlo tree search by best arm identification. In *Advances in Neural Information Processing Systems*, 2017. arXiv:1706.02986.

[Lan *et al.*, 2021] Li-Cheng Lan, Meng-Yu Tsai, Ti-Rong Wu, I-Chen Wu, and Cho-Jui Hsieh. Learning to stop: Dynamic simulation Monte-Carlo tree search. In *AAAI Conference on Artificial Intelligence*, 2021. arXiv:2012.07910.

[Leela Chess Zero contributors, 2026] Leela Chess Zero contributors. Lc0 options. <https://github.com/LeelaChessZero/lc0/wiki/Lc0-options>, 2026. Accessed 2026-10-01.

[Muppidi *et al.*, 2026] Aneesh Muppidi, Firas Darwish, Dylan Cope, João F. Henriques, and Jakob Nicolaus Foerster. Finding the time to think: Learning planning budgets in real-time RL. *arXiv preprint arXiv:2606.26463*, 2026.

[Silver *et al.*, 2016] David Silver, Aja Huang, Chris J. Maddison, Arthur Guez, Laurent Sifre, George van den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, Sander Dieleman, Dominik Grewe, John Nham, Nal Kalchbrenner, Ilya Sutskever, Timothy Lillicrap, Madeleine Leach, Koray Kavukcuoglu, Thore Graepel, and Demis Hassabis. Mastering the game of Go with deep neural networks and tree search. *Nature*, 529:484–489, 2016.

[Silver *et al.*, 2017] David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, Yutian Chen, Timothy Lillicrap, Fan Hui, Laurent Sifre, George van den Driessche, Thore Graepel, and Demis Hassabis. Mastering the game of Go without human knowledge. *Nature*, 550:354–359, 2017.

[Wang *et al.*, 2023] Tony T. Wang, Adam Gleave, Tom Tseng, Kellin Pelrine, Nora Belrose, Joseph Miller, Michael D. Dennis, Yawen Duan, Viktor Pogrebniaik, Sergey Levine, and Stuart Russell. Adversarial policies beat superhuman Go AIs. In *International Conference on Machine Learning*, pages 35655–35739, 2023. arXiv:2211.00241.

[Wu, 2019] David J. Wu. Accelerating self-play learning in Go. *arXiv preprint arXiv:1902.10565*, 2019.

[Wu, 2026] David J. Wu. KataGo opening books. <https://katagobooks.org/>, 2026. Accessed 2026-10-01.

[Xiao *et al.*, 2018] Chenjun Xiao, Jincheng Mei, and Martin Müller. Memory-augmented Monte Carlo tree search. In *AAAI Conference on Artificial Intelligence*, 2018.

[Ye *et al.*, 2022] Weirui Ye, Pieter Abbeel, and Yang Gao. Spending thinking time wisely: Accelerating MCTS with virtual expansions. In *Advances in Neural Information Processing Systems*, 2022. arXiv:2210.12628.